

Normal and Parallax Mapping in Three JS

Angela Gross // CSCI-441 // Project 2

INTRODUCTION

When creating an interactive 3D graphics program, mapping textures on 3D geometries can be an inexpensive way to add realism to a scene by adding color(s) and pattern(s) to the geometry. A texture is inexpensive to use due to each of the texture's pixels (or texels) being pre-computed and stored in a static image file. Although cheap to use, the realism added by textures is limited by the "flatness", or two dimensions, of the texture. This project seeks to emulate depth in a flat texture by using different kinds of texture mapping to emulate depth for a given texture, and to learn about the different kinds of texture mapping in order to choose which combinations would be best for the scene. Originally this project had a goal of placing a realistic looking terrain within the first project, (which used Rayleigh and Mie scattering to create a realistic sun and sky scene), but the shader for the sky unrealistically mapped the grass texture onto the sky and failed to light the terrain properly. Unable to fix this problem in a timely matter, the goal for this project shifted to simply using a procedurally generated terrain geometry that utilizes a vertex and fragment shader that implements normal and parallax mapping in order to add depth to the terrain's texture.

Normal and parallax mapping are inexpensive alternatives to a type of texture mapping that displaces each vertex on a geometry by using a height map that correlates to the texture¹. In other words, it physically deforms the geometry so light accurately scatters from the object. Displacement mapping results in the most realistic effect compared to normal and parallax mapping, but it is a good deal more expensive to use since it must displace every vertex². Normal mapping, on the other hand, uses a texture called a normal map that encodes the XYZ values of the surface in the RGB channels³. In order to calculate the surface normal to place in the RGB channels, it uses a normal map associated with the given diffuse texture. This encoded normal replaces (or, in our case, augments) the geometry's normal so the light and shadows calculated emulates depth. Parallax mapping, which is almost always used in conjunction with normal mapping, displaces the UV coordinate (which is a coordinate that keeps track of the texel) by using the diffuse and height maps⁴, and this results in making the texture look like it "pops out" even though no vertices are displaced. Since the combination of normal and parallax mapping give a decent approximation

¹ Premecz, Matyas. "Iterative Parallax Mapping with Slope Information." <http://www.cescg.org/CESCG-2006/papers/TUBudapest-Premecz-Matyas.pdf>.

² Karhu, Kimmo. "Displacement Mapping." *Academia.edu*. http://www.academia.edu/2696670/Displacement_Mapping.

³ Villar, Jacobo Rodriguez. "OpenGL Shading Course: Chapter 4 - Advanced Shaders." *OpenGL*. https://www.opengl.org/sdk/docs/tutorials/TyphoonLabs/Chapter_4.pdf.

⁴ Premecz, Matyas. "Iterative Parallax Mapping with Slope Information." <http://www.cescg.org/CESCG-2006/papers/TUBudapest-Premecz-Matyas.pdf>.

of depth and is not as expensive to use as displacement mapping, these techniques would be appropriate to use for the terrain especially since the terrain could potentially be extremely large. Thus, a description on how normal and parallax mapping works will need to be discussed.

Normal and parallax mapping computations are done in a space that is different from the light, view, and camera vectors because the surface normals in the normal maps are encoded in “tangent space”⁵ and the UV coordinates for parallax mapping correlate to tangent space as well⁶. Thus, we first need to define tangent space and how we can convert our vectors and values to tangent space before we discuss how normal and parallax mapping are computed. The use of tangent space is due to the fact that the normals remain unchanged when we rotate the geometry, but it would be changed if it remained in “model space”, or the space the light, view, and cameras lie in. These normals remain unchanged because tangent space is relative to each face of the geometry, and so the position and rotation of the geometry does not affect the normals in tangent space. Tangent space can be defined by creating an “orthonormal per-vertex basis”, and this basis will help us convert the relevant vectors from model to tangent space. In order to build this basis, we will need the normal, tangent, and binormal vectors for each vertex. The normal is perpendicular to the local surface, and the tangent and binormal vectors are perpendicular to the normal and are on the tangent plane on the local surface⁷, as Figure 1⁸ shows. The normal and tangent vectors are given from Three JS, which is the framework we are using. The computation of these vectors is outside of the scope of this project, but a discussion of these computations can be found online⁹. However, the

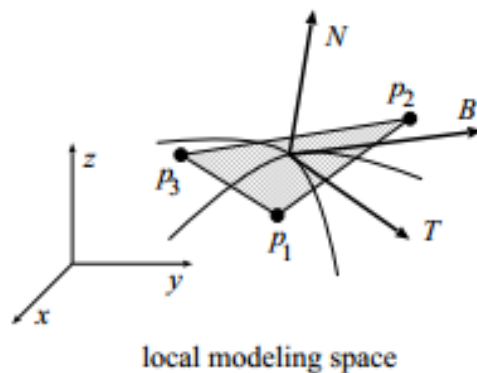


Figure 1: Normal, binormal, and tangent vectors in model space.

⁵ Villar, Jacobo Rodriguez. "OpenGL Shading Course: Chapter 4 - Advanced Shaders." *OpenGL*. https://www.opengl.org/sdk/docs/tutorials/TyphoonLabs/Chapter_4.pdf.

⁶ Premecz, Matyas. "Iterative Parallax Mapping with Slope Information." <http://www.cescg.org/CESCG-2006/papers/TUBudapest-Premecz-Matyas.pdf>.

⁷ Ibid

⁸ Ibid

⁹ <http://blog.db-in.com/calculating-normals-and-tangent-space/> and <http://www.cescg.org/CESCG-2006/papers/TUBudapest-Premecz-Matyas.pdf>.

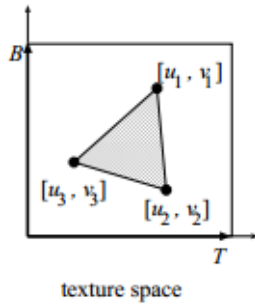


Figure 2: Visual correlation between tangent and binormal vectors to u, v coordinates.

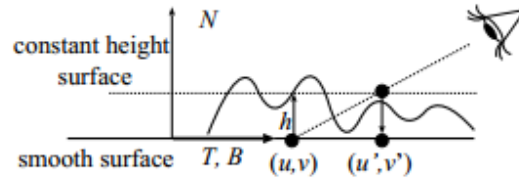


Figure 3: Graphical visualization of (u, v) and (u', v') coordinates, the viewing vector, and height at (u, v) in tangent space.

binormal is very easy to compute, and is simply $B = (T \times N)m$, where T is the tangent, N is the normal, and m designates the direction (or “handedness”) the binormal will point¹⁰. Once we have those vectors, then the basis can be defined as $[T \ B \ N]$, or a 3×3 matrix that is multiplied by a vector in model space to get a vector that is in tangent space. With tangent space computed, now we can easily discuss normal and parallax mapping.

Normal mapping, as mentioned earlier, changes the normal by using a static texture that stores the local surface normals which are stored in tangent space. The red, green, and blue channels correlate to the x , y , and z values (respectively) for the surface normals, and the RGB stores these by converting the normal values from $[0, 1]$ to $[-1 \text{ to } 1]$ ¹¹. Thus, to convert it back we multiply the RGB value by 2 and subtract by one, or:

$$\hat{n} = \text{normalMap}(u, v)_{rgb} * 2 - 1$$

Where u, v are the texel coordinates. From there, we continue on with the light calculations (making sure that the relevant vectors are in tangent space).

While normal mapping adjusts the normal, parallax mapping changes what u, v coordinates are used. Since normal mapping needs to use the u, v coordinates to sample the static normal map texture, the parallax mapping calculations need to be done before the normal mapping in order to use the correct normal from the normal map texture. Parallax mapping replaces the given u, v values by u', v' , which are computed by retrieving the height h at u, v and view direction V_{xyz} which are both in tangent space¹². The importance of being in tangent space is, as Figure 2¹³ shows, the T and B vectors that help define the tangent space basis correspond to the u and v values for a texel, respectively. Figure 3¹⁴ shows how these variables relate to one another, and helps develop a mental model for the problem. Assuming that the height in the surrounding area of u, v is constant, we can use similarity to see that:

¹⁰ Lengyel, Eric. “Computing Tangent Space Basis Vectors for an Arbitrary Mesh”. Terathon Software 3D Graphics Library, 2001. <http://www.terathon.com/code/tangent.html>

¹¹ DesLauriers, Matt. Lesson 6: Normal Mapping. <https://github.com/mattdesl/lwjgl-basics/wiki/ShaderLesson6>

¹² Premecz, Matyas. “Iterative Parallax Mapping with Slope Information.” <http://www.cescg.org/CESCG-2006/papers/TUBudapest-Premecz-Matyas.pdf>.

¹³ Ibid

¹⁴ Ibid

$$(u' - u, v' - v) : h = (V_x, V_y) : V_z$$

Thus we can see that (u', v') would be:

$$(u', v') = (u, v) + h * \left(\frac{V_x}{V_z}, \frac{V_y}{V_z} \right)$$

You can calculate the height value, h , by retrieving the height value from a texture. However, the stored values are in the range of $[0, 255]$, and the height may be negative and/or in the floating point¹⁵, and so it must be scaled and biased like so:

$$h = \text{heightMap}(u, v)_a * \text{scale} + \text{bias}$$

However, a strange defect occurs when you divide by the z-component of the viewing vector: the texture looks distorted when viewing at low or “grazing” angles. This is because of the original assumption that the height at u, v will be the same at u', v' , and that is rarely true. It could be true if the surface is viewed at near perpendicular angles, as the texture offset is small and the heights are nearly the same¹⁶. It is not true, however, when the surface is viewed at “grazing” angles, as the offset approaches infinity and the heights are not the same. Matyas Premecz in his paper, *Iterative Parallax Mapping with Slope Information*, suggests that omitting the division by V_z would be a good solution. However, the texture still appears to “slide” or “swim” across the surface, but this effect can be minimized with the correct scale and bias values. Once we have found the new u', v' coordinates, we use u', v' to sample from the diffuse and normal textures and continue on to our light calculations. Although the light calculations are extremely important in the context of computer graphics, it is not the focal point of this project, and so for more information on the lighting we used, (which is a Lambertian directional and ambient light), one can consult Edward Angel’s computer graphics text¹⁷.

With the understanding of how normal and parallax mappings work, we will now discuss the methods of our implementation.

METHODS

The implementation for this project was done within the framework of Three JS, a lightweight JavaScript library and API used to create 3D computer graphics within a web browser¹⁸, along with WebGL, a JavaScript API based on OpenGL for rendering aforementioned 3D graphics within compatible web browsers¹⁹. Utilizing a pre-existing Teapot Demo from Eric Haine’s Udacity course²⁰, a 3D scene was first set up by a file called *TerrainScene.js*, vertex and fragment shaders were added to an object that

¹⁵ Premecz, Matyas. "Iterative Parallax Mapping with Slope Information." <http://www.cescg.org/CESCG-2006/papers/TUBudapest-Premecz-Matyas.pdf>.

¹⁶ Ibid

¹⁷ Angel, Edward. 2009. *Interactive Computer Graphics: A Top-Down Approach Using OpenGL*. Pearson Education, Inc. Chapter 6.

¹⁸ "Three.js." GitHub. <https://github.com/mrdoob/three.js>.

¹⁹ "The Khronos Group." WebGL. <https://www.khronos.org/webgl/>.

²⁰ Haines, Eric. "Interactive 3D Graphics." Course With Three.js & WebGL. <https://www.udacity.com/course/cs291>.

resides in *TerrainShader.js*, and these shaders were used to create a material that was applied to a procedurally generated Three JS PlaneGeometry²¹ that is created with *TerrainGenerator.js*.

Although implementing *TerrainScene.js* is fairly trivial in the context of shader programming, it is still worth mentioning briefly how it functions. A Three JS Perspective Camera and WebGL Renderer are first created along with some camera controls that Haines developed, and Google's dat.GUI is added in order for the user to change the parameters that affect the shader uniform variables. Those are namely terrain roughness (for the terrain generation), light azimuth and inclination, light intensity, whether or not to use normal and/or parallax mapping, the parallax scale and bias values, and the different textures that you can use. The scene is then created which consists of adding the camera and generating the terrain. The algorithm that is used to generate the terrain in *TerrainGenerator.js* is called the diamond-square algorithm. An in-depth discussion of this algorithm is outside the scope of this project, but one can be found online²². The shaders that are used to modify the material that is applied to the generated terrain, however, is what we will now discuss.

TerrainShader.js, as mentioned before, is a container for an object that contains the uniform, vertex, and fragment shader variables. The uniforms are defined as stated in the Three JS wiki²³, and the vertex and fragment shaders are a concatenation of strings that represent the shader program, which is written in GLSL. There are other ways to read in shaders other than doing this (such as using jQuery's Ajax method to read in a shader file's text²⁴), but these methods are a little more complex and did not seem worth investigating at the time. The uniform variables are parallel to what is in the dat.GUI: texture2D samplers for the diffuse (color), normal, and height map textures; float values for the scale and bias for the parallax height values; integer values to be used as booleans to determine if normal and/or parallax mapping is used; a float for the light intensity; and 3-component vectors for the light's position and color. The vertex shader transforms the model coordinates to view coordinates for the view, light, and camera vectors, and converts those to tangent space and passes them (along with the UV coordinates) to the fragment shader via varying variables. The fragment shader uses the equations we defined in the introduction to displace the texture with parallax mapping, change the normal with normal mapping, and update the fragment color using a Lambertian illumination model. The implementation of these shaders is based off of these equations along with various tutorials, and so a discussion about the similarities and differences between those tutorials will be addressed.

Chapter 4 from an online course provided by OpenGL and Typhoon Labs called *OpenGL Shading Language Course* was helpful in my implementation of normal and parallax shaders, as they used GLSL to implement their shader. Another tutorial that assisted me with normal mapping along with creating a

²¹ "Three.js / Docs." Three.js. <http://threejs.org/docs/#Reference/Extras/Geometries/PlaneGeometry>

²² Martz, Paul. "Generating Random Fractal Terrain." Generating Random Fractal Terrain. <http://www.gameprogrammer.com/fractal.html>.

²³ "Uniform types." GitHub. <https://github.com/mrdoob/three.js/wiki/Uniforms-types>.

²⁴ "jQuery.ajax()." jQuery API Documentation. <http://api.jquery.com/jquery.ajax/>.

Shader	OpenGL	Matt D.	Angela Gross
Tangent Space Basis	<pre>// Precomputed basis attribute vec3 tangent; attribute vec3 binormal; attribute vec3 gl_Normal;</pre>	None	<pre>// Uses precomputed attributes to compute // binormal vNormal = normalize(normal); vTangent = normalize(tangent.xyz); vBinormal = normalize(cross(vNormal, vTangent) * tangent.w);</pre>
Tangent Space Conversion	<pre>// Doesn't convert to view lightvec *= TBNMatrix;</pre>	None	<pre>// Converts to view vLightPos = modelViewMatrix * lightPos; tsLightPos = TBN * vLightPos;</pre>
Normal	<pre>// Uses normal from map norm = texture2D(normalAndHeight, nUV).rgb * 2.0 - 1.0;</pre>	<pre>// Uses normal from map N = NormalMap * 2.0 - 1.0;</pre>	<pre>// Combines geometry and map normals normalMap = normalize(normalMap * 2.0 - 1.0); fNormal = blendWhiteout(tsNormal, normalMap);</pre>

Figure 4: Table of main differences between implemented shaders. (Code modified for brevity)

Lambertian light was titled *Lesson 6: Normal Mapping*, and originates from Matt DesLauriers' GitHub wiki pages. The first tutorial mostly assisted me with parallax mapping, as the second tutorial helped me the most with the normal mapping, and it helped explain how tangent space worked in a simple and direct way. However, the usage of their view, light, and camera vectors did not make sense as the vectors need to be converted to world coordinates before you convert to tangent space²⁵. From here we need to convert to view coordinates, otherwise the lighting would not change as the geometry/camera/light changed position. The second tutorial helped me understand how normal mapping works along with making a simple light, but it greatly failed in the implementation of normal mapping. They did not convert their relevant vectors into tangent space, and so their light calculations were not correct. Additionally, the generated terrain geometry is a bumpy plane, and, since the height and normal maps do not include this bumpiness, the normal from the normal map must be combined with the geometry's pre-existing normal. The method of determining how to combine these normals came from the research of computer graphics developers Colin Barré-Brisebois and Stephen Hill in a blog post, *Blending in Detail*²⁶, and seem to work well for this project. Again, the math behind the combinations of these normals is unrelated to the project, but Barré-Brisebois and Hill go into great detail about how it is derived. The comparisons between the project's shaders and the ones listed in all of the tutorials are provided in Figure 4.



Figures 5-7: Three different textures mapped onto the same procedurally generated terrain. The grass, dry ground, and rubber textures had their height and normal maps generated by Crazy Bump.

²⁵ Hegde, Siddharth. Messing With Tangent Space. http://www.gamasutra.com/view/feature/129939/messing_with_tangent_space.php.

²⁶ Hill, Colin, and Barré-Brisebois Stephen. Blending in Detail. <http://blog.selfshadow.com/publications/blending-in-detail/>.

Although a good deal of work is done in the vertex and fragment shaders, parallax and normal mapping heavily depends on the normal and height maps for a given texture, and it is possible to use a program to generate these. For this project, a program called *Crazy Bump*²⁷ was used in order create these maps for any texture. Since these normal and height maps are procedurally generated, the generated maps are sometimes inaccurate and fails to capture all of the height/normal information. However, *Crazy Bump* does a decent job of approximating this information, and, since it has a 30 day evaluation period and a discounted student license, it is one of the better alternatives to texture map generation. Figures 5 – 7 show the program using three different textures on a single terrain utilizing normal and parallax mapping. Note that the texture used in Figure 7 does not look very realistic on bumpy terrain since rubber does not warp that way naturally.

TESTING AND VERIFICATION

In order to make sure that each part created in *TerrainScene.js* and *TerrainShader.js* is working correctly, a unit testing framework was created to test every method in *TerrainScene.js* and compile the shaders in *TerrainShader.js*. The reader can use the unit tester themselves by opening up *tests/index.html*. These unit tests wrap the methods in a try-catch along with looking at the compile-status of WebGL but they only check if the program could run without any compile or run time errors. These tests do not check for any logical errors. However, the parts that need the logical testing are the vertex and fragment shaders in *TerrainShader.js*. Short of recreating a vertex shader, rasterizer, and fragment shader in another programming language, unit testing shaders in WebGL is extraordinarily difficult since there is not an easy way to debug a GLSL program and see what each shader is outputting. However, an examination of the source code along with cross-referencing papers to make sure that each part of the code is doing what it should be is a good examination that assures that the program is doing what it needs to. Since this necessitates listing both shaders in their entirety, this verification is done in separate document in *Gross_Angela_Project_2_Source_Code_Verification.pdf*.

ANALYSIS

We can further examine the functionality of the shader by investigating the relationships between the important variables in the fragment shader. We already have established the mathematical relationships between the variables in the introduction, and so we will use the dat.GUI provided in the program and examine the results visually. The examination will have a “control” state, or a state of the program that we can compare to. The default conditions for each of these variables in the control state

²⁷ "CrazyBump." CrazyBump. <http://www.crazybump.com/>.

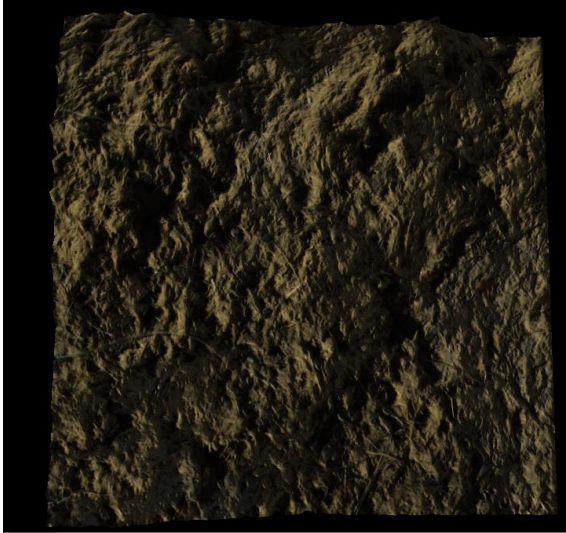


Figure 8: Photo of the control state.

terrainRoughness: 0.17
lightAzimuth: 0, lightInclination: 1, lightIntensity: 1
normal: true
parallax: true, scale: -0.003, bias: 0.001
texture: DryGround

The variables in red will not change during this examination



Figure 9: Terrain without normal mapping, but with parallax mapping

terrainRoughness: 0.17
lightAzimuth: 0, lightInclination: 1, lightIntensity: 1
normal: false
parallax: true, scale: -0.003, bias: 0.001
texture: DryGround



Figure 10: Terrain without normal or parallax mapping

terrainRoughness: 0.17
lightAzimuth: 0, lightInclination: 1, lightIntensity: 1
normal: false
parallax: false, scale: -0.003, bias: 0.001
texture: DryGround

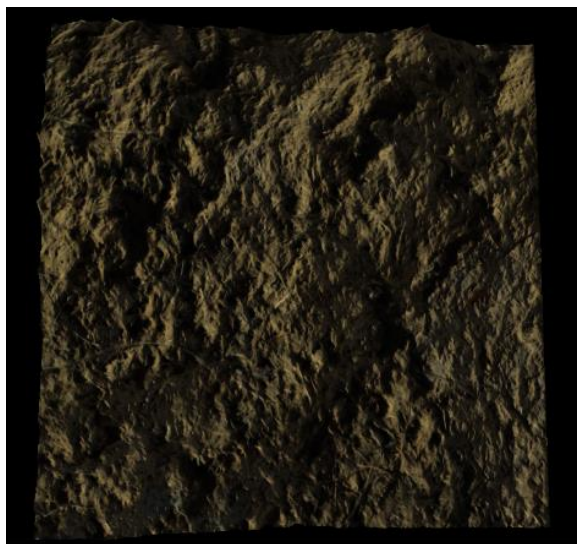


Figure 11: Normal mapping but no parallax mapping.

terrainRoughness: 0.17
lightAzimuth: 0, lightInclination: 1, lightIntensity: 1
normal: true
parallax: false, scale: -0.003, bias: 0.001
texture: DryGround

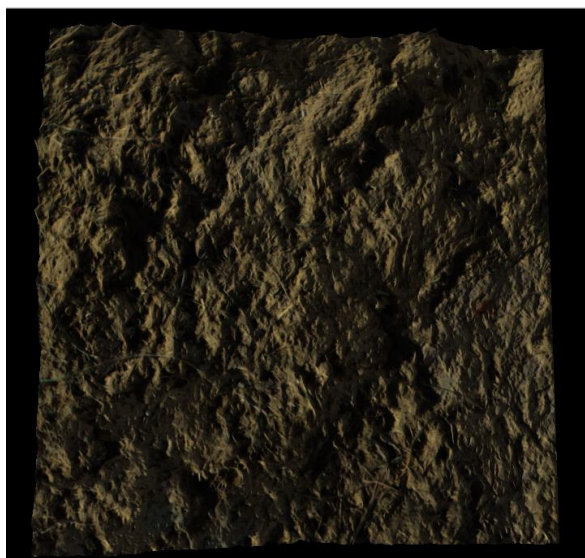


Figure 12: No value for parallax scale, 0.

terrainRoughness: 0.17
lightAzimuth: 0, lightInclination: 1, lightIntensity: 1
normal: true
parallax: true, scale: 0.000, bias: 0.001
texture: DryGround

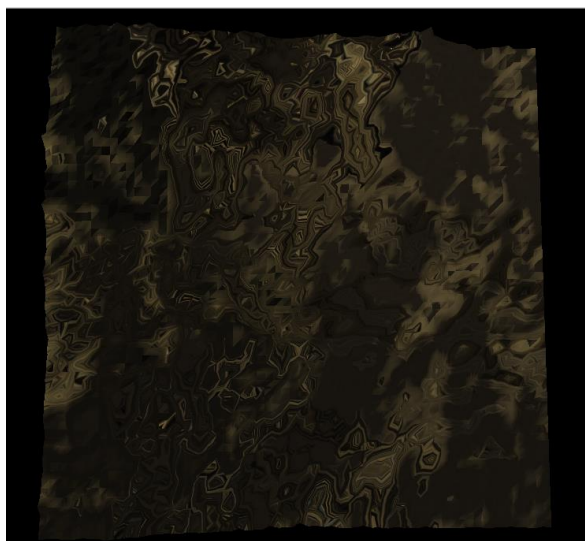


Figure 13: Extreme value for parallax scale, -1.

terrainRoughness: 0.17
lightAzimuth: 0, lightInclination: 1, lightIntensity: 1
normal: true
parallax: true, scale: -1.000, bias: 0.001
texture: DryGround

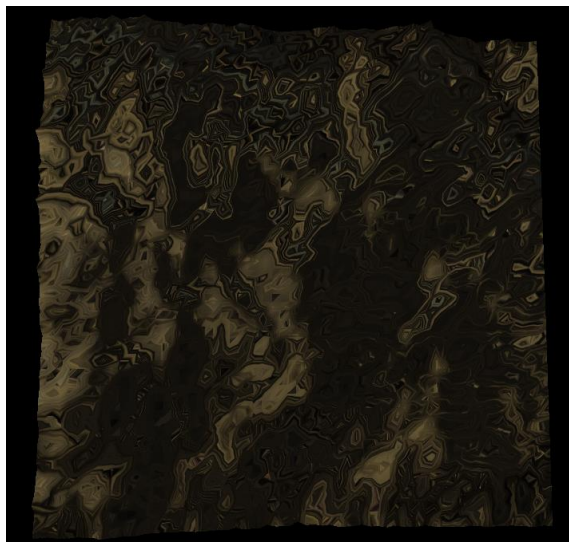


Figure 14: Extreme value for parallax scale, 1.

terrainRoughness: 0.17
lightAzimuth: 0, lightInclination: 1, lightIntensity: 1
normal: true
parallax: true, scale: 1.000, bias: 0.001
texture: DryGround

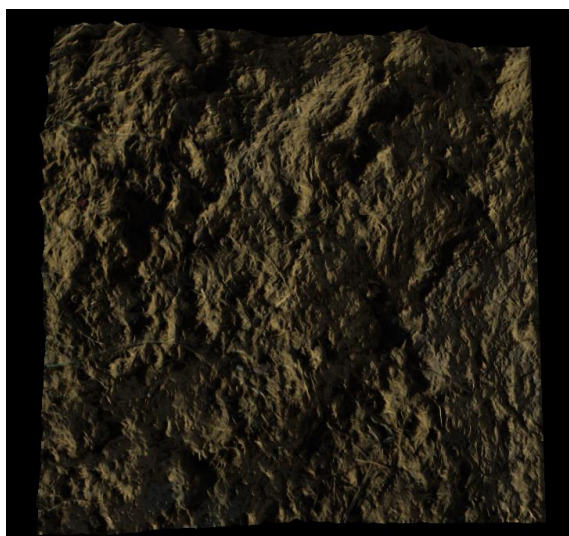


Figure 15: No value for parallax bias, 0.

terrainRoughness: 0.17
lightAzimuth: 0, lightInclination: 1, lightIntensity: 1
normal: true
parallax: true, scale: -0.003, bias: 0.000
texture: DryGround



Figure 16: Extreme value for parallax bias, -1.

terrainRoughness: 0.17
lightAzimuth: 0, lightInclination: 1, lightIntensity: 1
normal: true
parallax: true, scale: -0.003, bias: -1.000
texture: DryGround

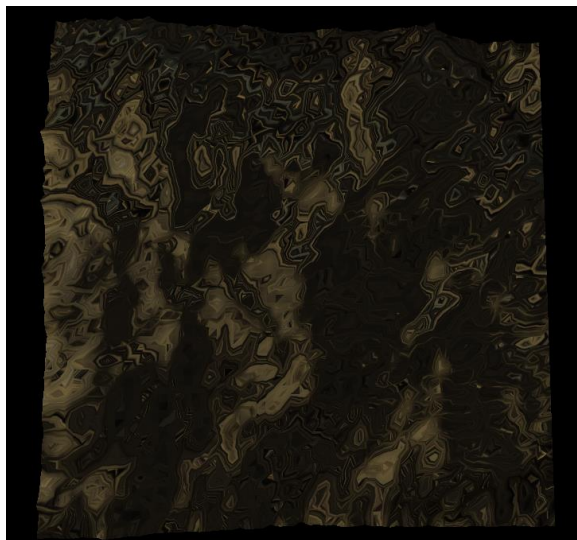


Figure 17: Extreme value for parallax bias, 1.

terrainRoughness: 0.17
lightAzimuth: 0, lightInclination: 1, lightIntensity: 1
normal: true
parallax: true, scale: -0.003, bias: 1.000
texture: DryGround

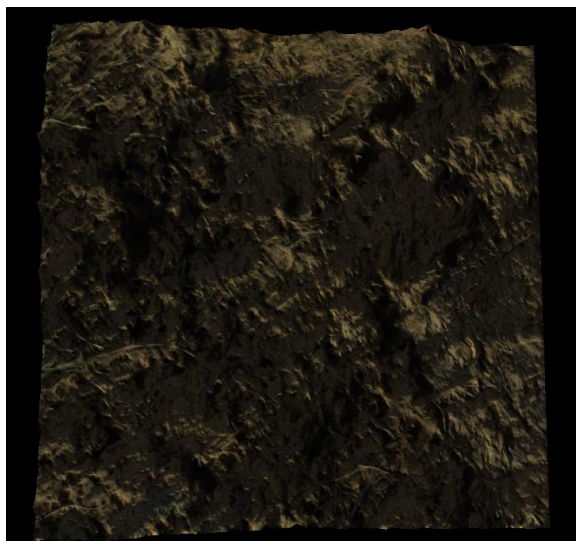


Figure 18: Small value for inclination, 0.2

terrainRoughness: 0.17
lightAzimuth: 0, lightInclination: 0.2, lightIntensity: 1
normal: true
parallax: true, scale: -0.003, bias: 0.001
texture: DryGround

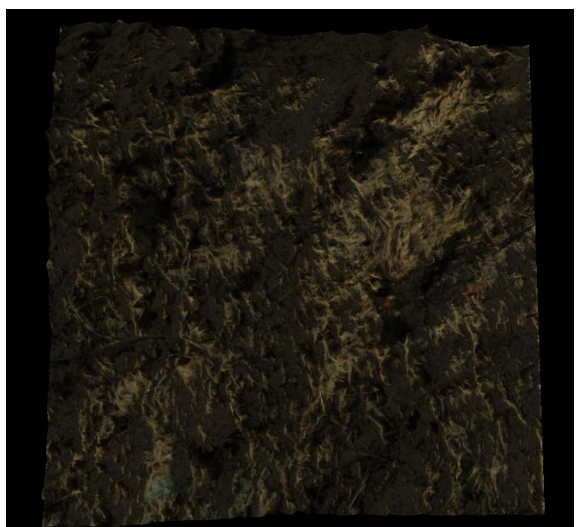


Figure 19: Moderate value for inclination, 0.6

terrainRoughness: 0.17
lightAzimuth: 0, lightInclination: 0.6, lightIntensity: 1
normal: true
parallax: true, scale: -0.003, bias: 0.001
texture: DryGround

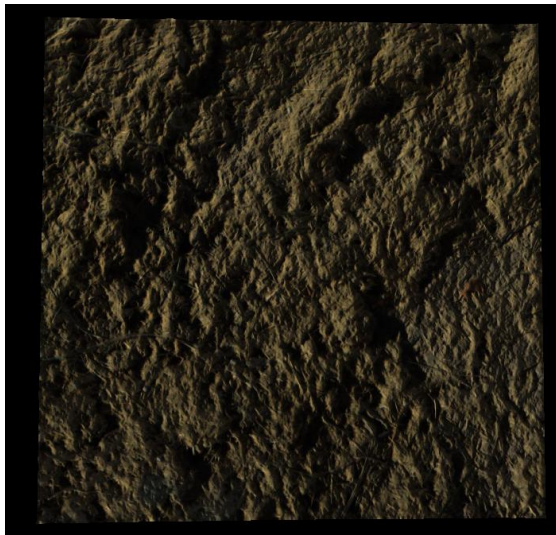


Figure 20: Flat terrain

terrainRoughness: 0.01
lightAzimuth: 0, lightInclination: 1.0, lightIntensity: 1
normal: true
parallax: true, scale: -0.003, bias: 0.001
texture: DryGround

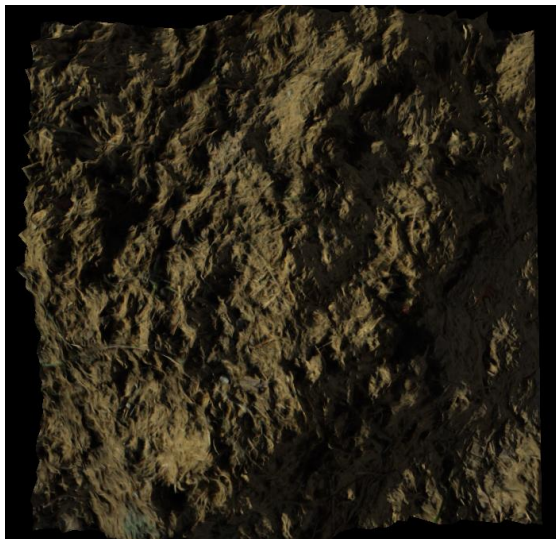


Figure 21: Fairly bumpy terrain

terrainRoughness: 0.36
lightAzimuth: 0, lightInclination: 1.0, lightIntensity: 1
normal: true
parallax: true, scale: -0.003, bias: 0.001
texture: DryGround

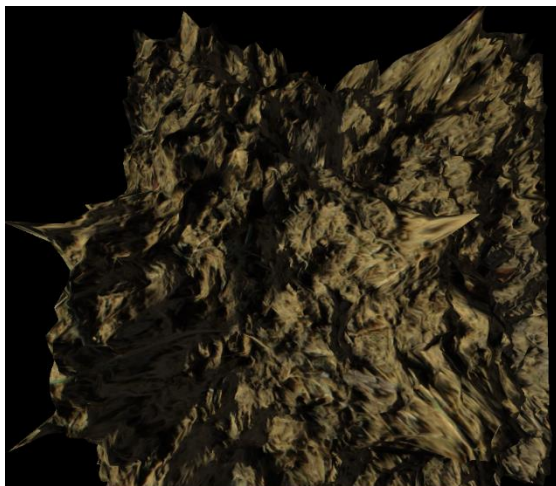


Figure 22: Extremely bumpy terrain

terrainRoughness: 0.99
lightAzimuth: 0, lightInclination: 1.0, lightIntensity: 1
normal: true
parallax: true, scale: -0.003, bias: 0.001
texture: DryGround

will be based on what looks “the best”, and will use the “dry ground” texture, as that texture creates a great deal of “bumps” and variety in heights. The variables that we will change will be the terrainRoughness, normal, parallax, scale, and bias. Aside from the light’s inclination, the varying in lighting will not be focused on since our focus is normal and displacement mapping and not illumination models. Figure 8 displays the control state’s variable values along with a detailed screen shot of the state. The figures following Figure 8 change the values for normal and parallax mapping booleans, scale, bias, light inclination, and terrain roughness.

Figures 9 – 11 show the state without normal mapping, without normal nor parallax mapping, and with normal but without parallax mapping. When parallax mapping is absent (or, in the inverse, when normal mapping is absent but parallax mapping is present) it seems like parallax mapping does not make a huge difference. This may be because the terrain texture is not “bumpy” enough to exploit parallax mapping, as many examples of parallax mapping have bumpy textures such as cobblestone. It could also be due to the scale and bias values not being large enough to distort the texture in a large way; the larger the scale and bias values, however, the larger the distortion. On the contrary, normal mapping makes a huge difference in light and shadow quality, and it greatly enhances the depth of the texture. Figures 12 – 17 demonstrate the changes in the texture when the scale and bias values are changed. As a reminder, the scale and bias variables scale and offset the height given from the displacement map at texel u,v to help with the computation of the new texel u',v' in parallax mapping. The scale and bias values are fairly critical to one another, and, if one is not balanced with the other, the texture looks almost like liquid. Since parallax mapping on its own does not change the texture very noticeably, the absence of scale or bias seems to not affect the texture as well. Figures 18 – 19 demonstrate the difference of lighting effects when the inclination of the light source is changed. This reveals other rough parts of the terrain that we would not have seen if the light remained stationary. Note that the lighting in Figure 18 seems to create the illusion that there are pebbles or other small objects on the terrain. Finally, Figures 20 – 22 shows the different kinds of roughness the terrain generator can produce. We can see in Figure 20 that the flat terrain still shows an incredible amount of depth simply because of the normal and parallax mapping and seems fairly realistic. Figure 22, on the other hand, distorts the texture far too much to seem very realistic, but shading still seems in tact.

INTERPRETATION

All in all, the project was a success even though the terrain did not make it into the previous project’s sky dome. The project’s main goals were to research and implement normal and parallax mapping along with displacement mapping, and the majority of those goals were met. However, due to time constraints along with worrying about the expensive computations associated with displacement mapping, displacement mapping was only researched and not implemented. Additionally, the limited

effect that parallax mapping could provide to a given terrain, (without distorting the texture), along with the “swimming” defect on the texture, was disappointing. In spite of that, normal mapping, along with the combination of the geometry’s surface normals with the methods from Colin Barré-Brisebois and Stephen Hill²⁸, gave the terrain a great deal of depth in an inexpensive way. In addition, even though the parallax mapping did not apply well to the terrain, the “swimming” and “liquid” effects from parallax mapping could be exploited to create a water texture. This could be used to help finish off the landscape scene that these projects are striving to create. Thus, all of the discoveries made in this project, good or bad, were beneficial and will be able to benefit projects in the future.

²⁸ Hill, Colin, and Barré-Brisebois Stephen. Blending in Detail. <http://blog.selfshadow.com/publications/blending-in-detail/>.

BIBLIOGRAPHY

- Angel, Edward. 2009. *Interactive Computer Graphics: A Top-Down Approach Using OpenGL*. Pearson Education, Inc. Chapters 6 and 9.
- Bomfim, Diney. n.d. *Calculating Normals and Tangent Space*. <http://blog.db-in.com/calculating-normals-and-tangent-space/>.
- Chadwick, Rob. n.d. *Terrain Editor*. <http://robchadwick.com/TerrainEditor/>.
- DesLauriers, Matt. n.d. *Lesson 6: Normal Mapping*. <https://github.com/mattdesl/lwjgl-basics/wiki/ShaderLesson6>.
- Haines, Eric. n.d. *Interactive 3D Graphics*. <https://www.udacity.com/course/cs291>. Lesson 8.
- Hegde, Siddharth. n.d. *Messing With Tangent Space*. http://www.gamasutra.com/view/feature/129939/messing_with_tangent_space.php.
- Hill, Colin, and Barré-Brisebois Stephen. n.d. *Blending in Detail*. <http://blog.selfshadow.com/publications/blending-in-detail/>.
- McGuire, Morgan. n.d. "Steep Parallax Mapping." <http://graphics.cs.brown.edu/games/SteepParallax/mcguire-steeparallax.pdf>.
- n.d. *Moon WebGL*. http://whiteflashwhitehit.com/content/2011/02/moon_webgl.html.
- mrdoob. n.d. *Three.js*. <https://github.com/mrdoob/three.js>.
- n.d. *Pixel Normals*. <http://mattdesl.github.io/kami-demos/release/normals-pixel.html>.
- Premecz, Matyas. n.d. "Iterative Parallax Mapping with Slope Information." *CESCG*. <http://www.cescg.org/CESCG-2006/papers/TUBudapest-Premecz-Matyas.pdf>.
- Villar, Jacobo Rodriguez. n.d. "OpenGL Shading Course: Chapter 4 - Advanced Shaders." *OpenGL*. https://www.opengl.org/sdk/docs/tutorials/TyphoonLabs/Chapter_4.pdf.
- n.d. *Web GL Map Demo*. http://threejs.org/examples/webgl_materials_normalmap.html.